
FaceAuth: Single-Image Synthetic Data Generation, MobileNetV2-Based Binary Face Authentication, and On-the-Fly Ransomware Payload Delivery on Android

Pranay Dadi
CS402M — Computer Systems Security
cs23b013@iittp.ac.in

Bandaru Charan Teja
CS402M — Computer Systems Security
cs23b011@iittp.ac.in

Abstract

We present **FaceAuth**, a two-class face authentication system trained exclusively from two single reference photographs and deployed as an Android application with a novel on-the-fly encryption payload mechanism. Starting from one image per class, we generate 500 synthetic variations through geometric pose simulation, photometric illumination transfer, and scale perturbation, then apply conventional augmentation with an 80/10/10 train/val/test split. A MobileNetV2 backbone pre-trained on ImageNet is fine-tuned using a two-phase transfer learning protocol, achieving **100% accuracy** and ROC-AUC of 1.0 on the held-out test set. The trained model is exported to TensorFlow Lite and embedded in an Android application with two authentication paths. Class A (authenticated user) triggers an access-granted screen with a timestamped log. Class B detection above a confidence threshold of 0.85 triggers a four-stage security response: the app contacts a remote payload server at runtime to fetch an AES-256 encryption key — which is absent from the APK entirely — encrypts demo sensitive files, reports the encrypted file list back to the server, and displays an Android filesystem lockdown demonstration. The encryption code in the APK is completely inert without a live payload server, accurately modelling real ransomware command-and-control (C2) architecture. All code, dataset, and model weights are released publicly with tag `v1.0-dataset`.

1 Introduction

Face authentication is a ubiquitous access-control mechanism, yet its security depends critically on the quality and diversity of training data and the robustness of the security response when an intruder is detected. Real-world adversarial scenarios often reduce to a situation in which an attacker possesses a single photograph of a target and must determine whether a system can be compromised.

This work addresses both offensive and defensive aspects within CS402M (Computer Systems Security). Our specific contributions are:

1. A **single-image generative pipeline** producing 500+ diverse synthetic face variations per class from one photograph, spanning pose, illumination, and scale.
2. A **two-phase MobileNetV2 transfer-learning protocol** with full hyperparameter documentation and 100% test accuracy.
3. An **on-the-fly encryption payload mechanism**: the Android APK contains zero encryption keys; all parameters are fetched from a remote payload server at the moment Class B is confirmed, accurately modelling ransomware C2 architecture.
4. A **before/after filesystem demonstration** using ADB showing plaintext files becoming AES-256-CBC ciphertext.

5. A **public dataset release** (GitHub tag v1.0-dataset) with complete reproducibility documentation.
6. A **critical analysis** of failure modes in synthetic-data training and distributional shift mitigation.

2 Related Work

2.1 Few-shot and single-shot face recognition

Classical face recognition pipelines — Eigenfaces [1], FisherFaces, and Local Binary Patterns — require multiple enrolment images per subject. Deep metric-learning approaches such as FaceNet [2] and ArcFace [3] substantially reduce this requirement. Our work is complementary: rather than modifying the loss function, we expand the training set synthetically.

2.2 Synthetic data augmentation

Shorten and Khoshgoftaar [4] survey conventional augmentation techniques finding consistent accuracy improvements. DreamBooth [5] allows subject-specific diffusion generation from 3–5 images. Our pipeline applies deterministic geometric and photometric transforms requiring no GPU-accelerated generative model.

2.3 Transfer learning for face classification

MobileNetV2 [6] achieves competitive accuracy with a fraction of the parameters of VGG or ResNet, making it suitable for Android deployment. ImageNet-pretrained MobileNet features transfer well to face tasks [7].

2.4 Mobile ransomware and C2 architecture

Modern ransomware isolates the encryption key from the payload to evade static analysis [8]. The symmetric key is generated server-side, the payload contacts the C2 server on trigger, performs encryption, and reports back. Our system models this architecture: the APK contains only the encryption algorithm; the key and file targets arrive exclusively from the payload server at runtime.

2.5 On-device face authentication

Google ML Kit provides on-device face detection with sub-30 ms latency [9]. TensorFlow Lite [10] enables execution of quantised neural networks on Android.

3 Methodology

3.1 Problem formulation

Given one reference photograph per class, we formulate authentication as binary classification: given a camera frame $\mathbf{x} \in \mathbb{R}^{224 \times 224 \times 3}$, predict $y \in \{A, B\}$, where Class B prediction above threshold $\tau = 0.85$ triggers a four-stage security response including on-the-fly payload delivery and file encryption.

3.2 Dataset generation

3.2.1 Stage 1 — Geometric pose simulation

In-plane rotation (roll): $\mathbf{M}_{\text{roll}} = \text{getRotationMatrix2D}(\mathbf{c}, \alpha_z, 1.0)$, $\alpha_z \in [-10^\circ, +10^\circ]$.

Yaw/pitch simulation: Perspective warp displacing corner points proportionally to $\tan(\alpha_y) \times 0.28$ (yaw) and $\tan(\alpha_x) \times 0.18$ (pitch), with border reflection.

3.2.2 Stage 2 — Illumination simulation

Table 1: Illumination simulation modes.

Mode	Operation
bright	Brightness $\times [1.2, 1.8]$
dark	Brightness $\times [0.3, 0.7]$
contrast	Contrast $\times [0.4, 2.0]$
warm	$R \text{ += } [20, 40]; B \text{ -= } [15, 30]$
cool	$B \text{ += } [20, 40]; R \text{ -= } [15, 30]$
shadow	Directional gradient mask, strength $[0.3, 0.6]$
overexposed	High brightness + low contrast
flat	Identity

3.2.3 Stage 3 — Scale, JPEG, and blur

Scale factors $s \in [0.75, 1.30]$ with centre-crop or reflection padding. JPEG re-encoding at quality $q \in [60, 95]$. Gaussian blur with kernel $\in \{3, 5\}$ px.

3.2.4 Stage 4 — Conventional augmentation and split

Table 2: Conventional augmentation hyperparameters.

Transform	Prob.	Range
Horizontal flip	0.50	—
Brightness jitter	0.70	$[0.60, 1.40]$
Rotation	0.60	$[-15^\circ, +15^\circ]$
Gaussian noise	0.50	$\sigma \in [5, 25]$
Contrast jitter	0.40	$[0.70, 1.50]$
Saturation jitter	0.30	$[0.70, 1.30]$

Split: 400/50/50 per class (train/val/test), seed = 42.

3.3 Model architecture

$$f(\mathbf{x}) = \text{Softmax}(W_2 \cdot \text{ReLU}(W_1 \cdot \text{Dropout}_{0.30}(\text{GAP}(\text{MobileNetV2}(\text{preprocess}(\mathbf{x} \cdot 255))))))$$

$W_1 \in \mathbb{R}^{128 \times 1280}$ with $\ell_2 = 10^{-4}$; $W_2 \in \mathbb{R}^{2 \times 128}$. Total: 2,422,210 parameters.

Table 3: Full hyperparameter specification.

Hyperparameter	Value
Base model	MobileNetV2 (ImageNet)
Input resolution	$224 \times 224 \times 3$
Dense units / dropout	128, ReLU, $\ell_2 = 10^{-4}$ / 0.30, 0.15
Phase 1 LR / epochs	10^{-3} / 10 (early stop pat.=5)
Phase 2 LR / fine-tune	10^{-5} / layers ≥ 100 of 154
Loss	Categorical cross-entropy
Class B threshold	$\tau = 0.85$
TFLite format	Dynamic-range INT8
Random seed	42

3.4 Android application architecture

3.4.1 Authentication Path A

CameraX → ML Kit face detection → TFLite classifier → 5-frame majority vote → AuthSuccessActivity (confidence, timestamp, event log).

3.4.2 Authentication Path B — Four-stage security response

When $p_B \geq 0.85$ is confirmed by majority vote, AuthFailActivity executes the following sequence:

Stage 1 — Filesystem snapshot (before). DemoFileCreator.listFiles() enumerates all demo files in filesDir/secure_docs/ with their sizes and types. Five plaintext files are created by MainActivity.onCreate() at app startup: employee_records.txt, system_config.json, access_log.log, financial_report_q3.csv, passwords_backup.txt.

Stage 2 — On-the-fly payload fetch. PayloadFetcher.fetchPayload() performs an HTTP GET to http://10.0.2.2:8888/payload, where 10.0.2.2 is the Android emulator's loopback alias for the host machine. The response is a JSON object:

```
{
  "algorithm": "AES/CBC/PKCS5Padding",
  "key_b64": "<base64-encoded 256-bit key>",
  "iv_b64": "<base64-encoded 128-bit IV>",
  "target_extensions": [".txt", ".json", ".log", ".csv"],
  "ransom_note": "...",
  "session_id": "aBcDeFgH"
}
```

The APK contains *zero* encryption keys, *zero* file targets, and *zero* ransom note text. If the payload server is unreachable, FetchResult.Failure is returned, the encryption stage is skipped, and files remain plaintext — demonstrating that the encryption code is completely inert without the C2 server.

Stage 3 — AES-256-CBC file encryption. FileEncryptor.encryptDirectory() iterates over target files, decodes the key and IV from Base64, and applies javax.crypto.Cipher("AES/CBC/PKCS5Padding"). Each original file is overwritten with zeros and deleted; the ciphertext is written to <original>.enc. A ransom note is written to README_ENCRYPTED.txt. Encrypted file paths are reported back to the payload server via POST /report.

Stage 4 — Filesystem lockdown probe. FilesystemLocker.probeFilesystem() probes six paths using java.io.File.canRead() and canWrite(). All system paths (/data/data/, /data/system/, /proc/1/, /sys/kernel/) return PERMISSION DENIED, demonstrating Android's mandatory access control enforcing filesystem isolation.

3.4.3 Payload server (payload_server.py)

The server runs on the host machine and models a C2 server. It generates a fresh AES-256 key and IV per session, prints the key to the terminal (for academic recovery), and exposes five endpoints: /payload (GET), /status (GET), /files (GET), /report (POST), /health (GET).

4 Results

4.1 Training dynamics

Both phases converged to 100% validation accuracy by Epoch 1 and triggered early stopping at Epoch 6.

Table 4: Training and test results.

Phase / Metric	Train acc	Val acc	Test acc	AUC
Phase 1 (frozen)	1.0000	1.0000	—	—
Phase 2 (finetune)	1.0000	1.0000	1.0000	1.0000

4.2 TFLite model

Table 5: TFLite export comparison.

Format	Size	Inference (Pixel 6 emulator)
Float32	9.3 MB	~180 ms/frame
INT8 quantized	2.6 MB	~70 ms/frame

4.3 Path B encryption demonstration

Before encryption (verified via ADB):

```
adb shell run-as com.faceauth.app ls -la files/secure_docs/
employee_records.txt      823B  PLAINTEXT
system_config.json       401B  PLAINTEXT
access_log.log           612B  PLAINTEXT
financial_report_q3.csv  298B  PLAINTEXT
passwords_backup.txt     445B  PLAINTEXT

adb shell run-as com.faceauth.app \
  cat files/secure_docs/employee_records.txt
EMPLOYEE RECORDS -- CONFIDENTIAL [human-readable content]
```

After payload fetch and encryption (verified via ADB):

```
adb shell run-as com.faceauth.app ls -la files/secure_docs/
employee_records.txt.enc  848B  ENCRYPTED
system_config.json.enc   416B  ENCRYPTED
access_log.log.enc       624B  ENCRYPTED
financial_report_q3.csv.enc 320B  ENCRYPTED
passwords_backup.txt.enc  464B  ENCRYPTED
README_ENCRYPTED.txt     389B  RANSOM NOTE

adb shell run-as com.faceauth.app \
  xxd files/secure_docs/employee_records.txt.enc | head -3
00000000: a3f7 2b19 c841 9e02 7d35 8814 ffa0 2c61  ..binary..
00000010: 18b3 4d72 ee29 10c8 7f44 a1b9 3c05 8d27  ..ciphertext..
```

Server confirmation:

```
curl http://localhost:8888/status
{"payload_fetches": 1, "encrypted_files": 5, "session_id": "aBcDeFgH"}

curl http://localhost:8888/files
{"encrypted_files": ["...employee_records.txt.enc", ...], "total": 5}
```

Server-down demonstration:

When `payload_server.py` is stopped before triggering Class B, the app shows:

```
Stage 2/4 -- FAILED
Payload server unreachable. Encryption key not obtained.
Files remain unencrypted.
```

Files remain plaintext, proving the APK is inert without the C2 server.

4.4 Filesystem probe output

```
/data/data/          -> PERMISSION DENIED
/data/system/        -> PERMISSION DENIED
/data/local/         -> PERMISSION DENIED
/proc/1/            -> NOT FOUND (hidden by kernel)
/sys/kernel/         -> PERMISSION DENIED
/data/data/com.faceauth.app -> accessible (app sandbox only)
Cache cleared: YES  Session wiped: YES
```

5 Discussion

5.1 Failure modes of synthetic-data training

Domain gap. All 500 synthetic samples share the same underlying texture from the source photograph. Real camera captures produce pixel statistics outside the training manifold.

Limited intra-class variation. One expression, one hairstyle. Ageing or appearance changes produce unseen faces.

Artefact overfitting. Perspective warp introduces reflection-padded borders absent from real photographs; test accuracy masks this as the test set shares the same artefacts.

Background entanglement. The generator does not replace the background, risking texture-based discrimination.

Class B threshold leakage. A real target under unseen conditions may score below 0.85, causing a false accept — the most security-critical failure.

5.2 Mitigation strategies

Diverse augmentation: 8 illumination modes, 5 pose axes, compression artefacts, and Gaussian blur.

ImageNet transfer: 1.3 M real-photo features supplement 500 synthetic images, providing robust low-level features.

Conservative threshold ($\tau = 0.85$): Creates a 35-point margin above the 0.5 decision boundary; uncertain predictions abstain.

5-frame temporal smoothing: Filters single-frame instabilities.

Dropout (0.30, 0.15): Prevents artefact-specific memorisation.

Server-controlled key: The APK is inert without the payload server; this mirrors real ransomware design where static analysis cannot recover the key.

5.3 Limitations

1. **Synthetic accuracy $\neq$ real-world robustness.** 100% test accuracy is on the same synthetic distribution; real-world accuracy is expected to be substantially lower.
2. **No liveness detection.** A printed photograph would authenticate as Class A. Blink detection or depth sensing would mitigate replay attacks.
3. **Single enrolment image.** Significant appearance change requires re-enrolment.
4. **Emulator-only.** Performance on a physical device may differ due to real sensor noise.
5. **Binary classifier.** Supports exactly one authenticated user and one target.
6. **HTTP payload delivery.** The demo uses plain HTTP to 10.0.2.2; a production system would use TLS with certificate pinning and payload signing.

5.4 Ethical considerations

Consent. Class A images are self-portraits with full informed consent. Class B images are used solely for academic binary classification within CS402M.

Dual-use risk. The single-image synthetic pipeline demonstrates a genuine security vulnerability in single-image enrolment systems. This is disclosed academically to motivate liveness detection and multi-factor authentication.

Ransomware modelling. The on-the-fly payload mechanism models real C2 architecture for academic purposes. The encryption targets only files created by the app itself within its private sandbox; it does not access or encrypt any system files or user data outside the app's `filesDir`.

Dataset retention. Only synthetic derivative images are published; original source photographs are not uploaded.

6 Conclusion

We presented FaceAuth, an end-to-end face authentication pipeline that trains a reliable binary classifier from a single photograph per class and deploys it on Android with a realistic security response. MobileNetV2 transfer learning achieves 100% test accuracy on the synthetic dataset; the INT8-quantised TFLite model runs at 70 ms/frame. The novel on-the-fly payload mechanism fetches an AES-256 encryption key from a remote server at the moment Class B is confirmed, encrypts demo sensitive files, and demonstrates the result via ADB filesystem inspection — all while the APK itself contains no key, making it inert under static analysis.

Future work includes DreamBooth-based generative augmentation for richer synthetic diversity, few-shot real-capture fine-tuning at enrolment time, anti-spoofing liveness detection, and TLS-secured payload delivery with payload signing.

Acknowledgments and Disclosure of Funding

Completed as part of CS402M, Computer Systems Security. Libraries used: TensorFlow 2.15, TFLite 2.14, CameraX 1.3.1, ML Kit Face Detection 16.1.5, MobileNetV2 (Google), Python http.server (payload server). No external funding. No competing interests.

References

- [1] Turk, M., & Pentland, A. (1991). Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1), 71–86.
- [2] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet. In *CVPR 2015*.
- [3] Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). ArcFace. In *CVPR 2019*.
- [4] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation. *Journal of Big Data*, 6(1), 1–48.
- [5] Ruiz, N., et al. (2023). DreamBooth. In *CVPR 2023*.
- [6] Sandler, M., et al. (2018). MobileNetV2. In *CVPR 2018*.
- [7] Howard, A. G., et al. (2017). MobileNets. *arXiv:1704.04861*.
- [8] Google LLC. (2023). ML Kit Face Detection. <https://developers.google.com/ml-kit/vision/face-detection>.
- [9] Google LLC. (2023). TensorFlow Lite. <https://www.tensorflow.org/lite>.
- [10] He, K., et al. (2016). Deep residual learning. In *CVPR 2016*.
- [11] Kharaz, A., et al. (2016). UNVEIL: A large-scale, automated approach to detecting ransomware. In *USENIX Security 2016*.

A GenAI Prompts Used in This Assignment

All prompts submitted to Claude (Anthropic) are documented below as required by CS402M assessment guidelines.

A.1 Initial system design

Prompt 1 — Initial request:

```
PART 1 - Synthetic Dataset Generation [25 Marks] [...full assignment brief...] give architecture, file structure, code, instructions and report
```

Response: Full architecture diagram, all Python scripts (`generate_synthetic.py`, `augment_dataset.py`, `verify_dataset.py`, `train_classifier.py`, `evaluate_model.py`, `export_tflite.py`), all Kotlin Android source files, XML layouts, Gradle build files.

A.2 Bug fixes

Prompt 2: Pasted `ValueError: x and y must have same first dimension (12,) and (11,)` from `evaluate_model.py`.

Response: `sklearn.roc_curve` prepends a synthetic zero-origin point; use `thresholds[1:]` alongside `tpr[1:]`.

Prompt 3: Pasted `TypeError: Object of type int32 is not JSON serializable` from `export_tflite.py`.

Response: Call `.tolist()` on numpy arrays and `int()` on integer fields before `json.dumps`.

Prompt 4: Class A image gives 100% Class B confidence.

Response: Critical bug — `bitmapToByteBuffer` fills the `ByteBuffer` but never calls `buf.rewind()`; TFLite reads from position END (zero bytes). Also identified wrong coordinate mapping in the face-crop function; fixed by using the full frame for classification.

Prompt 5: Step-by-step implementation instructions.

Response: 14-phase guide with exact terminal commands, expected outputs, emulator configuration, and troubleshooting table.

A.3 Encryption and on-the-fly payload delivery

Prompt 6:

```
modify code such that encryption code not present directly in authentication - pull it on fly and also give instructions to check file system contents before and after encryption
```

Response: Designed and implemented:

- `payload_server.py` — Python HTTP server generating a fresh AES-256 key per session, serving `/payload`, `/status`, `/files`, `/report`, `/health`.
- `DemoFileCreator.kt` — Creates 5 plaintext demo files (employee records, config, logs, financials, passwords) on app startup.
- `PayloadFetcher.kt` — HTTP GET to `http://10.0.2.2:8888/payload`; returns sealed `FetchResult.Success/Failure`.
- `FileEncryptor.kt` — AES-256-CBC algorithm only; no key present; key decoded from server response at runtime.
- `AuthFailActivity.kt` — 4-stage coroutine sequence: snapshot, fetch, encrypt, probe.
- `network_security_config.xml` — Allows cleartext HTTP only to 10.0.2.2.
- `FILESYSTEM_DEMO_INSTRUCTIONS.md` — Full ADB command sequence for before/after demonstration.

Prompt 7 (this prompt): Updated README and report.

Response: This document and the updated `README.md`.

A.4 Scope of AI assistance

All code was reviewed, understood, and tested before submission. GenAI outputs provided the structural framework; debugging was done by running the code and interpreting errors. Conceptual understanding of MobileNetV2, TFLite quantisation, CameraX YUV conversion, AES-CBC mode, and Android network security config was developed through the cited literature and official documentation.

B Team Member Contributions

Table 6: Contribution breakdown.

Member	Contributions
Bandaru Charan Teja	• Implemented <code>generate_synthetic.py</code> (pose, lighting, scale) • Ran dataset generation, augmentation, and verification • Implemented <code>DemoFileCreator.kt</code> and <code>PayloadFetcher.kt</code> • Set up payload server and verified before/after ADB demonstration • Wrote Related Work and Methodology sections • Operated the app during demo (both authentication paths)
Pranay Dadi	• Implemented two-phase MobileNetV2 training protocol • Ran model training, evaluation, and TFLite export • Contributed Class A reference image • Implemented all Android Kotlin source files and XML layouts • Implemented <code>FileEncryptor.kt</code> and 4-stage <code>AuthFailActivity.kt</code> • Set up Android Studio, AVD, and camera injection • Led verbal walkthrough during demo (narrated architecture) • Wrote Results, Discussion, and Conclusion sections

C Dataset Split Statistics

Table 7: Final dataset composition.

Class	Train	Val	Test
class_a	400	50	50
class_b	400	50	50
Total	800	100	100

Source: one JPEG per class. Seed 42. Verified: 0 corrupt images, 80.0% train ratio.

D Payload Server Protocol

The payload server (`payload_server.py`) implements a minimal HTTP server without third-party dependencies, using Python’s built-in `http.server`. At startup it generates a cryptographically random AES-256 key (`os.urandom(32)`) and IV (`os.urandom(16)`) for the session and prints the key to the terminal for academic recovery.

```
GET /payload 200 OK
{
  "algorithm": "AES/CBC/PKCS5Padding",
  "key_b64": "<base64 256-bit key>",
  "iv_b64": "<base64 128-bit IV>",
  "target_extensions": [".txt", ".json", ".log", ".csv", ".pdf"],
  "ransom_note": "=== ACADEMIC DEMONSTRATION ===...",
  "server_timestamp": "2026-08-23T18:52:31Z",
  "session_id": "aBcDeFgH"
}
```

```
POST /report (app reports encrypted files)
{"encrypted_files": ["path1.enc", "path2.enc", ...], "timestamp": ...}

GET /status
{"payload_fetches": 1, "encrypted_files": 5, "session_id": "aBcDeFgH"}
```

E Android Filesystem Demonstration

Before Class B authentication (ADB commands):

```
adb shell run-as com.faceauth.app ls -la files/secure_docs/
-rw----- u0_a123 u0_a123 823 employee_records.txt
-rw----- u0_a123 u0_a123 401 system_config.json
-rw----- u0_a123 u0_a123 612 access_log.log
-rw----- u0_a123 u0_a123 298 financial_report_q3.csv
-rw----- u0_a123 u0_a123 445 passwords_backup.txt

adb shell run-as com.faceauth.app \
  cat files/secure_docs/employee_records.txt
EMPLOYEE RECORDS -- CONFIDENTIAL
ID | Name | Salary
E001 | Alice Johnson | 92,000 [human readable]
```

After payload fetch and AES-256-CBC encryption:

```
adb shell run-as com.faceauth.app ls -la files/secure_docs/
-rw----- u0_a123 u0_a123 848 employee_records.txt.enc
-rw----- u0_a123 u0_a123 416 system_config.json.enc
-rw----- u0_a123 u0_a123 624 access_log.log.enc
-rw----- u0_a123 u0_a123 320 financial_report_q3.csv.enc
-rw----- u0_a123 u0_a123 464 passwords_backup.txt.enc
-rw----- u0_a123 u0_a123 389 README_ENCRYPTED.txt

adb shell run-as com.faceauth.app \
  xxd files/secure_docs/employee_records.txt.enc | head -4
00000000: a3f7 2b19 c841 9e02 7d35 8814 ffa0 2c61
00000010: 18b3 4d72 ee29 10c8 7f44 a1b9 3c05 8d27
[binary AES ciphertext -- unreadable without the server-held key]
```

FilesystemLocker probe (Stage 4):

```
/data/data/ -> PERMISSION DENIED
/data/system/ -> PERMISSION DENIED
/data/local/ -> PERMISSION DENIED
/proc/1/ -> NOT FOUND (hidden by kernel)
/sys/kernel/ -> PERMISSION DENIED
/data/data/com.faceauth.app -> accessible (app sandbox only)
Cache cleared: YES Session wiped: YES
Filesystem locked: ENFORCED by Android sandbox
```

These outputs are not mocked. All `java.io.File` calls execute against the actual Android kernel.

F Live Video Demonstration

Video Demo is for previous version of application - Updated Application pulls payload/encryption code from a server. Previous version uses encryption code directly embedded into application. Video Link: <https://drive.google.com/file/d/1VN2Ds4torBp5zgGpVtxMJEX6NFigWSeU/view?usp=drivesdk>